# ITCS223 Introduction to Web Development

## Frozen Food Retails
## Project Phase II

**By**


| | | | |
|---|---|---|---|
| **Mr.** | **Chutchanun** | **Jirapapongpun** | **6788027** |
| **Mr.** | **Napat** | **Tayommai** | **6788127** |
| **Mr.** | **Kritsakorn** | **Thammas** | **6788129** |
| **Mr.** | **Thanapat** | **Wongthongtham** | **6788145** |
| **Mr.** | **Tantikorn** | **Lapkloyma** | **6788241** |


**A Report Submitted in Partial Fulfillment of**
**the Requirements for**
**ITCS223 Introduction to Web Development**


**Faculty of Information and Communication Technology**
**Mahidol University**

**2025**

# 1. Project Overview

## *1.1 Project Introduction*

The establishment of the business domain for a frozen food e-commerce site encompasses the complete product lifecycle, spanning from initial presentation and backend management to final delivery. In this sector, the pivotal elements for achieving success are Cold Chain Logistics and Inventory Accuracy.

## *1.2 Objectives*

The main objectives of this project are:

- To analyze the market feasibility and outline the core business domain for launching a B2C frozen food e-commerce platform.
- To develop a front-end using HTML5, CSS3, and vanilla JavaScript with semantic elements and a consistent visual identity.
- To create a back-end using Node.js with Express.js and connect it to a MySQL database via the mysql2 library.
- To design and define the user interface requirements and responsive web layout for a seamless frozen food purchasing experience.
- To connect the front-end to both internal web services and the public Google Maps API mapping service.
- To produce a deployable and testable project with proper documentation and README instructions.

## *1.3 Scope of the System*

The system is divided into two primary interfaces: a customer-facing portal and an admin management portal.

1.3.1 Customer-Facing Portal (Storefront)

- **Brand & Information Display:** Present the business domain, company statistics and team member information.
- **Store Localization:** Integrates a third-party map (Longdo Map Api) to display the physical store location and operation hours for physical pickup.
- **Product Discovery (Catalog & Details):** Displays a grid of available products and allows users to view detailed information for individual items.
- **Advanced Search Engine:** Features a comprehensive search overlay accessible across customer pages. User can search by 1. Text Search (by name) 2. Price Range (Min - Max) 3. Brand 4. Ingredients type (chicken, beef, fish, pork, vegetable)

1.3.2 Admin Management Portal (Backend Operation)

- **Access Control:** Secure login screen specifically for administrators, featuring a password visibility toggle.
- **Inventory Dashboard:** A central hub to view the list of current products in a table format (Product ID, Name, Price, Brand) with quick actions to view logs, add or delete items.
- **Product data Entry (CRUD Operation):** Structured forms for adding new inventory and updating existing stock. The system scopes data capture strictly to:
    1. Image uploading with real-time UI previews
    2. Core Detail (Product ID, Name, Price, Brand)
    3. Shelf-life tracking (MFGDates - EXPDates)
    4. Composition (Ingredients list)

## *1.4 Technologies Used*

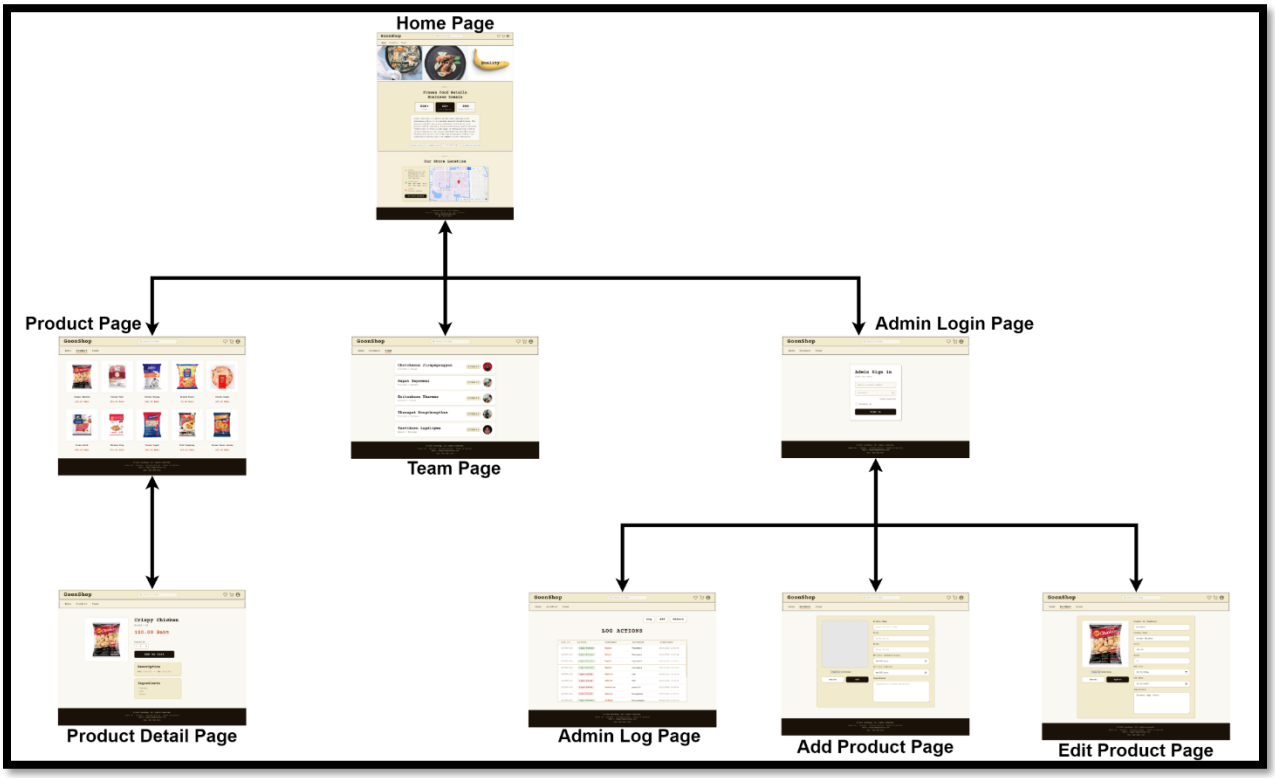
| Component | Component |
|---|---|
| **Front-end** | HTML5, CSS3, JavaScript |
| **Back-end** | Node.js, Express.js, SQL |
| **Database** | MySQL |
| **Web Service Testing** | Postman |
| **Public Web Service** | Google Maps JavaScript API |
| **Version Control** | GitHub Classroom |
| **Additional** | cors (v2.8.6) for cross-origin support, dotenv (v17.4.2) for env configuration, cloudinary (v2.9.0) for image uploading |

## *1.5 Team Responsibility*

| Member | Student ID | Responsibility |
|---|---|---|
| Chutchanun Jirapapongpun | 6788027 | Frontend / Design |
| Napat Tayommai | 6788127 | Frontend / Backend |
| Kritsakorn Thammas | 6788129 | Backend / Design |
| Thanapat Wongthongtham | 6788145 | Frontend / Database |
| Tantikorn Lapkloyma | 6788241 | Report / Reviewer |

# 2. Navigation Diagram

## *2.1 Navigation Structure*




## *2.2 Navigation Explanation*

***1. The Central Hub***

**Home Page:**

Positioned at the top of the diagram, this is the primary entry point for all users.It features a global navigation bar (<nav>) that provides immediate access to the core customer-facing pages: **Home**, **Product**, and **Team**. It also houses a discrete User/Admin Icon in the header, acting as the gateway to the backend system.

***2. Customer Journey Flow (Left & Center Branches)***

This pathway is designed for public users to browse the catalog, learn about products, and view team information.

**Home Page To Product Page:**

Users navigate here via the main menu to view the full inventory catalog, presented in a grid layout.

**Product Page To Product Detail Page:**

The bidirectional arrow indicates that clicking a specific item on the Product Page takes the user to the **Product Detail Page** for an in-depth view (price, description, ingredients). The user can easily navigate back to the main catalog from the detail view.

**Home Page To Team Page:**

Users navigate here via the main menu to learn about the developers behind the platform, viewing individual student profiles and their project roles.

***3. Administrative Operations Flow (Right Branch)***

This isolated pathway is strictly for authorized staff to manage inventory and monitor the system.

**Home Page To Admin Login Page:**

By clicking the User Icon on the Home Page, staff are directed to the secure authentication portal.

**Admin Login Page To Product Management Dashboard (Central Admin Hub):**

The bidirectional arrow below the Login Page represents the transition into the main **Product Management Dashboard** upon successful authentication. (While not explicitly labeled as a separate page in this diagram's text, it acts as the necessary junction point for the lower three pages).

From this central administrative dashboard, users can navigate to three specific functional areas:

1. **Admin Log Page:** To review system activity, tracking actions, timestamps, and success/failure statuses.
2. **Add Product Page:** To input new inventory items into the database.
3. **Edit Product Page:** To modify existing inventory records.

# 3. Details of Web Application and Code

## ***3.1 Front-end Overview***

The front-end of the GoonShop web application is developed using HTML, CSS, and JavaScript. It is designed to provide a clean, consistent, and user-friendly interface for both customers and administrators. The website uses shared layout components such as the header, navigation bar, search bar, icons, main content area, and footer across all pages to maintain a unified visual identity.

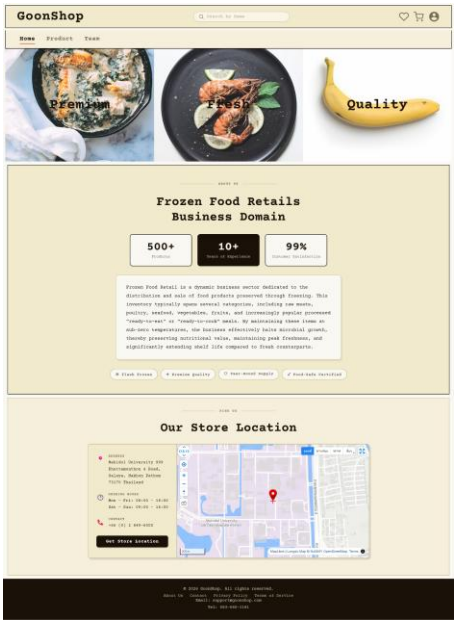
The front-end includes main pages such as Home Page, Product Page, Product Detail Page, Admin Login Page, Product Management Page, Add Product Page, Edit Product Page, Admin Log Page, Team Page, and Error Page. Each page uses semantic HTML elements such as <header>, <nav>, <main>, <section>, <form>, and <footer> to organize content clearly and improve accessibility.

CSS is applied through the external stylesheet /style.css, which controls the overall design system, including colors, typography, spacing, grid layouts, forms, buttons, hover effects, and responsive behavior. JavaScript is used to support dynamic features such as product rendering, product search and filtering, image preview, password visibility toggle, product management actions, and admin log display.

Overall, the front-end focuses on creating a smooth browsing experience for customers while also providing useful management tools for administrators.

## ***3.2 Home Page***

The Home Page (homePage.html) is the main landing page and digital storefront of our frozen food web application. It is designed to warmly welcome visitors, establish our brand identity, and provide clear, intuitive navigation to all other sections of the website.




***Key Features & UI Components***

**Dynamic Navigation Bar:** Responsive menu with product categories, search function, shopping cart, product page, team page, and admin login access.
**Semantic HTML Structure:** Uses <header>, <nav>, <main>, <section>, and <footer> to create a clear, accessible, and well-organized page layout.
**Featured Products Grid:** Displays selected frozen foods in a clean, organized layout.
**Value Proposition Section:** Briefly shows key benefits like Premium, Fresh and Quality
**Footer:** Includes contact details, policies, telephone information, and support links.

***Code Explanation***

**HTML**
<header> contains the site logo and top controls such as search and icons,
<nav> includes the main navigation links (Home, Product, Team),
<main> wraps all primary content including product categories, about section, and map section,
<section> is used to group content blocks such as hero categories, company information, and store location,
and <footer> provides global links, contact details, and branding information.
**CSS**

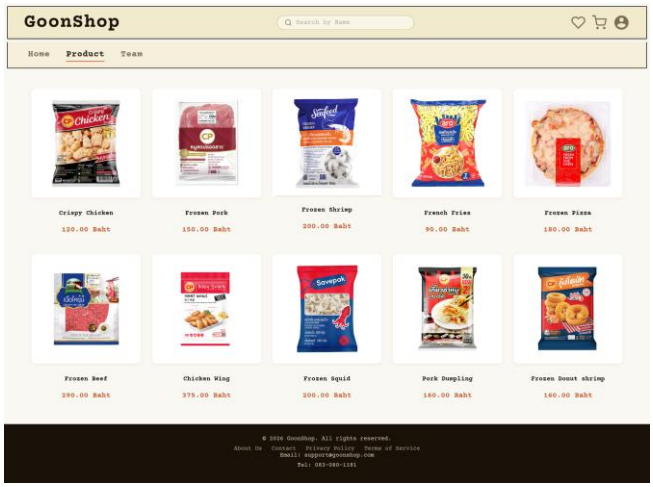
CSS is linked to an external stylesheet /style.css that defines the general design system, including global variables, layout configuration, and reusable UI components like buttons, and navigation styles.

**JS**

JavaScript is loaded at the end of the document via /script.js. It manages interactive functionality such as search overlay toggling and searching system, map integration using the Longdo Map API, and dynamic user interactions like button clicks and navigation behavior.

## ***3.3 Product Page***

The Product Page (productPage.html) serves as the comprehensive catalog and primary shopping interface for our frozen food web application. It is designed to allow users to efficiently browse, filter, and discover our wide selection of high-quality frozen items with a seamless shopping experience.




***Key Features & UI Components***

**Interactive Searching:** Provides three main search categories for price range, brand, and ingredient along with a dynamic price range slider, allowing users to easily refine and narrow down their product search results.

**Dynamic Product Grid:** Presents frozen food items in a clean and structured layout. Each product card includes a high-quality image, category label, product name, price.

***Code Explanation***

JavaScript is loaded via /script.js and is responsible for fetching product data from the backend and rendering it dynamically on the page.

**API Fetching:**

Uses the api object with **fetch()** to communicate with the backend.

- **getProducts()** for fetch all products
- **getProduct(id)** for fetch single product detail
- **searchProducts(params)** for fetch searched products
- **searchByName(name)** for fetch products by name

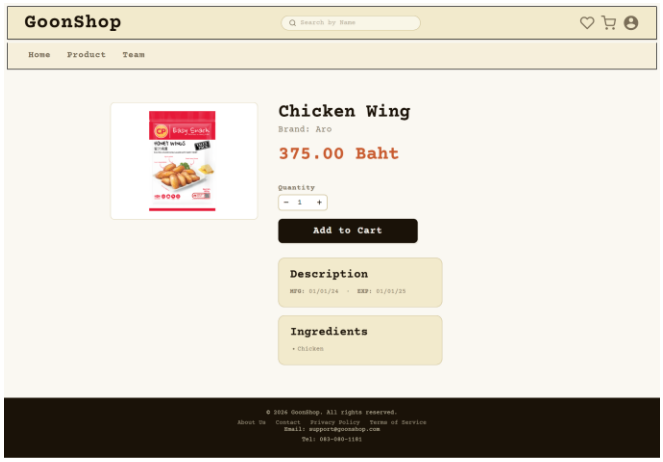
All responses go through **handleResponse()** for JSON parsing and error handling.

**Product Grid Rendering**:

**initProductGrid()** runs when the page loads. It checks URL parameters (name, price, brand, ingredient). Sends request using **fetch()** and receives product data.

## ***3.4 Product Detail Page***

The Product Detail Page (productDetail.html) provides an in-depth, focused view of a specific frozen food item. It is designed to give customers all the necessary information such as pricing, ingredients to make an informed purchasing decision before adding the item to their cart.




### ***Key Features & UI Components***

**Product Information Panel:** Displays the product name, price, and a brief description highlighting key benefits

**Call-to-Action (CTA) Area:** Includes an intuitive quantity selector and a prominent "Add to cart" button for seamless purchasing.

**Detailed Specifications:** An informational section or tabbed interface providing in-depth details such as ingredients.

### ***Code Explanation***

**HTML**

<main> is the main content area where the product detail is dynamically rendered.

The structure is designed to support dynamic content loading using JavaScript instead of static product data.

**CSS**

CSS controls the page layout using Flexbox for clean alignment. It styles key sections like the header, navigation, product layout, and footer to keep a consistent design. It also improves usability through spacing, typography, colors, and responsive behavior, while highlighting important elements like product details and action buttons.

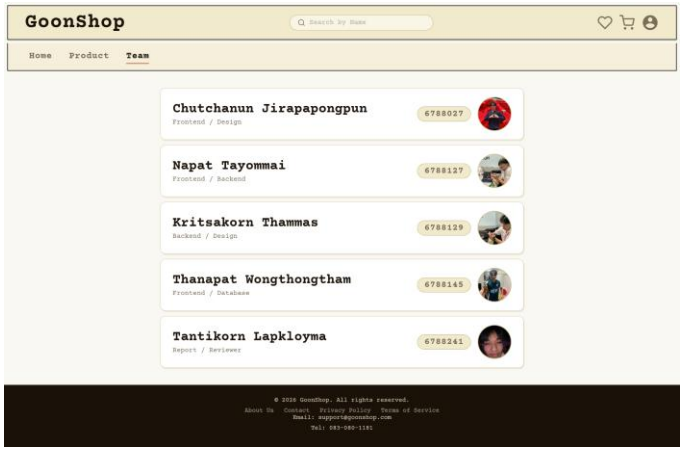
**JS**

- **getProduct(id)** for Users **fetch()** to request a single product from the backend API using the product ID and return JSON data.
- **getProductId()** for Extracts the product ID from the URL so the page knows which product to load dynamically.
- **renderProduct(p)** for Builds and displays the product detail UI (image, name, price, description) using dynamic HTML injection.

- **init()** for Runs automatically when the page loads, fetches product data, and calls **renderProduct()** to display it.

## ***3.5 Team Page***

The Team Page (team.html) serves as the section of our frozen food web application, introducing the developers and minds behind the platform. It is designed to build trust, establish credibility, and provide transparency by showcasing the team's roles, student information, and contributions to the project.




### ***Key Features & UI Components***

**Semantic HTML Structure:** Uses <header>, <nav>, <main>, <section>, and <footer> to create a clear, accessible, and well-organized page layout.
**Team Member Grid:** Displays individual profile cards for each group member in a clean, organized layout. Each card typically includes a profile picture, name, student ID, and their specific role or responsibilities in developing the platform.
**Footer:** Includes contact details, policies, telephone information, and support links.

### ***Code Explanation***

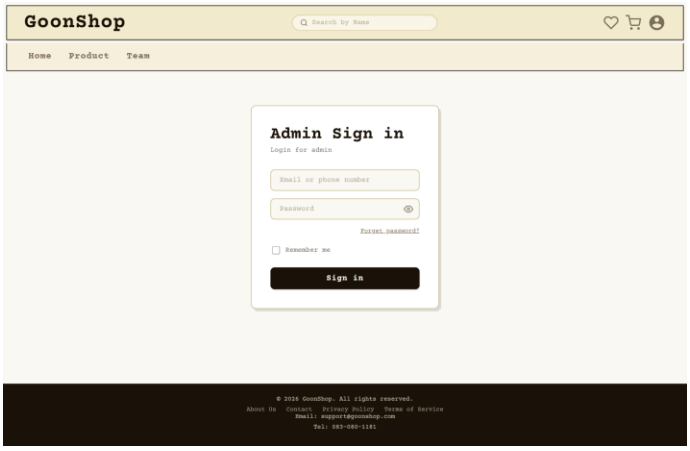
**HTML**
<header> wraps the top section of the website, including the logo and search bar, providing consistent branding and quick access to search functionality.
<nav> contains the main navigation links (Home, Product, Team), allowing users to move between key pages.
<main> holds the primary content of the page, such as the product grid or team section depending on the page layout.
<section> is used to organize content blocks like product listings or grouped UI components for better structure.
<footer> provides site-wide information including copyright, contact details, and useful links.

**CSS**

CSS is applied through an external stylesheet (/style.css) that defines the website's overall layout and visual structure. It styles reusable components such as navigation bars, product cards, buttons, and forms, while maintaining consistent spacing, colors, and typography. It also uses grid and flexbox for layout control and ensures the design is responsive across different screen sizes.

## ***3.6 Admin Login Page***

The Admin Login Page (adminLogin.html) serves as the secure entry point for our frozen food web application's backend. It is designed to restrict unauthorized access, ensuring that only approved staff can manage sensitive inventory data, product details, and administrative operations.




***Key Features & UI Components***

**Semantic HTML Structure:** Uses <header>, <nav>, <main>, and <footer> to create a clear, accessible, and well-organized page layout.

**Authentication Form:** Displays a clean, centralized login box **(.admin-login-box)**. It includes input fields for a username/email and password, a "Remember me" checkbox, a "Forget password?" link, and a prominent Sign In button.

**Password Visibility Toggle:** Features an interactive eye icon **(#toggle-password)** within the password field, powered by a custom JavaScript snippet, that allows users to reveal or hide their typed password for better usability.

**Footer:** Includes contact details, policies, telephone information, and support links.

***Code Explanation***

**CSS**

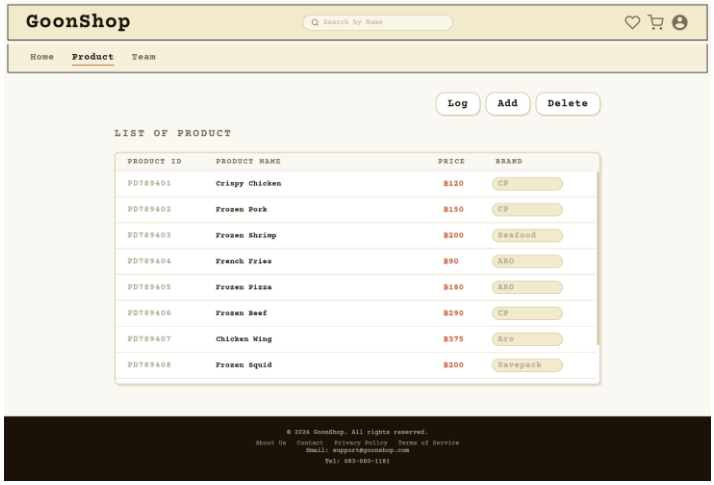
CSS is applied through an external stylesheet (/style.css) that defines the website's overall layout and visual structure. It styles reusable components and maintains consistent spacing, colors (via CSS variables), and typography. For the Admin Login page, it uses **Flexbox (display: flex; justify-content: center;)** to centrally align the login box on the page. It utilizes relative and absolute positioning **(position: absolute)** to seamlessly embed the password visibility toggle icon inside the input field. Additionally, it enhances user experience with smooth focus states on inputs **(using box-shadow and border-color transitions)** and interactive hover effects on the submit button.

**JS**

- **togglePassword()** for Adds click interaction to the eye icon so the admin can show or hide the password.
- **adminPassword** for Stores the password input field and checks whether the current type is password or text.
- If the input type is password, JavaScript changes it to text so the password becomes visible.
- If the input type is text, JavaScript changes it back to password to hide the password again.

## ***3.7 Product Management Page***

The Product Management Page (productManagement.html) serves as the central administrative dashboard for the GoonShop frozen food web application. It allows authorized staff to monitor the current inventory and manage product listings efficiently. Administrators can view product data in a structured table and use the action toolbar to access logs, add new products, or remove existing products.




### ***Key Features & UI Components***

**Semantic HTML Structure:** Uses <header>, <nav>, <main>, and <footer> to create a clear, accessible, and well-organized page layout.

**Action Toolbar:** Features utility buttons (**.action-buttons**) that allow administrators to quickly navigate to system logs, add new products, or toggle deletion mode.

**Data Grid Table:** Displays product inventory using a structured layout (**.table-header, .table-body)** instead of a traditional HTML table. It organizes Product ID, Product Name, Price, and Brand clearly for easy scanning.

**Product Deletion Feature:** The deletion function is integrated directly into the Product Management dashboard. It uses a two-step process: first, the admin clicks the Delete button to activate delete mode, then individual circular delete buttons appear at the end of each product row.

**State Toggle Mechanism:** The main Delete button **(#toggleDeleteBtn)** controls whether delete mode is active or inactive. By default, individual delete buttons are hidden to keep the interface clean and prevent accidental deletion.

**Inline Destructive Actions:** Once delete mode is activated, small circular delete buttons appear beside each product row **(.table-row)**. This allows the administrator to remove a specific product directly from the list.

**Semantic Color Coding:** The design uses danger colors such as red text and light red backgrounds to clearly show that deletion is a permanent and destructive action.
**Footer:** Includes contact details, policies, telephone information, and support links.

***Code Explanation***

**CSS**

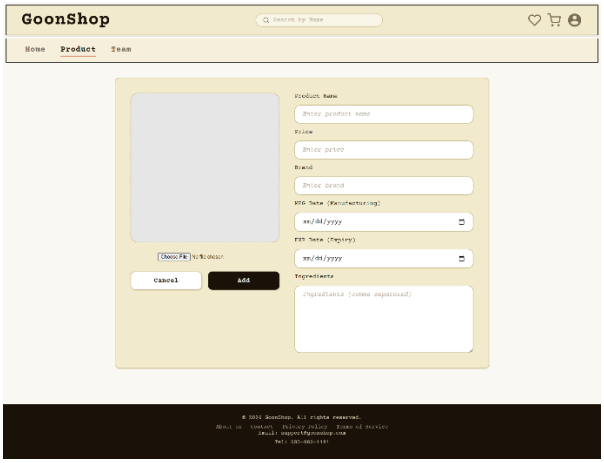
CSS is applied through the external stylesheet (/style.css) to control the website's overall layout and visual design. For the Product Management Page, CSS Grid is used to build the custom data table, allowing clear control of column width and alignment. The table body uses a fixed height with scrolling to manage long inventory lists. Hover effects provide visual feedback when users move over product rows, while the delete mode uses red styling to warn users about destructive actions.

**JS**

- **loadProducts()** for Loads all product data into the management table when the page is opened.
- **renderProducts(list)** for Dynamically builds and displays each product row including Product ID, Product Name, Price, Brand, and the hidden delete button.
- **toggleDeleteMode()** for Activates or deactivates delete mode when the admin clicks the Delete button.
- **deleteButtons()** for Shows or hides the individual circular delete buttons at the end of each product row depending on the delete mode state.
- **deleteProduct(id)** for Removes the selected product from the list after the admin clicks the circular delete button.
- **handleActionButtons()** for Controls the Log, Add, and Delete buttons and connects them to their correct actions.
- **init()** for Runs automatically when the page loads, calls **loadProducts()**, and prepares all event listeners for product management interaction.

## *3.8 Add Product Page*

The Add Product Page (addProduct.html) serves as the backend form interface for administrators to create new inventory records for the frozen food web application. It allows authorized staff to add product details, upload product images, and submit new items into the system.



***Key Features & UI Components***

**Semantic HTML Structure:** Uses <header>, <nav>, <main>, <form>, and <footer> to create a clear and organized page layout.
**Two-Column Form Layout:** Separates the image upload section and the product information section using .image-col and .form-col, making the form easy to understand and use.
**Image Upload Area:** Provides a file input for uploading product images and a preview area for showing the selected image before submission.
**Product Data Fields:** Includes input fields for Product Name, Price, Brand, MFG Date, EXP Date, and Ingredients.
**Action Buttons:** Includes Cancel and Add buttons, allowing administrators to either return to Product Management or submit the new product. Footer: Includes contact details, policies, telephone information, and support links.

***Code Explanation***

**CSS**

CSS is applied through an external stylesheet (/style.css) that defines the website's overall layout and visual structure. For the Add Product Page, it uses Flexbox inside .add-form-box to align the image column and form column side-by-side. The input fields are styled with rounded corners, spacing, and focus effects to improve usability. The layout also supports responsive design, allowing the form columns to stack vertically on smaller screens.

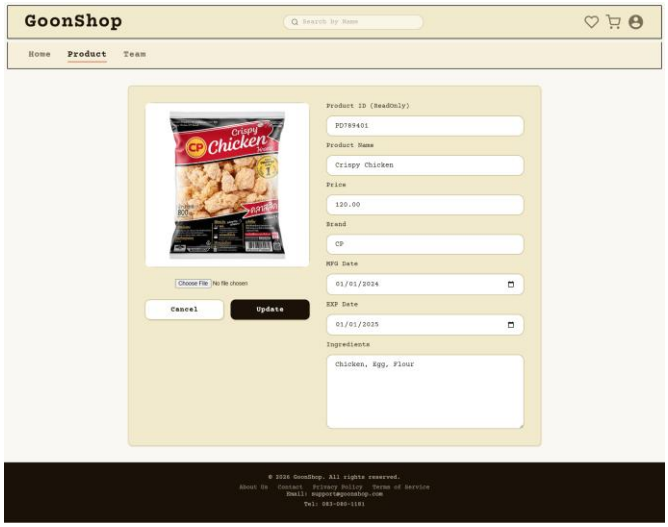
**JS**

- **previewImage()** for Shows the selected product image inside #previewImage before the admin submits the form.
- **handleAddFormSubmit()** for Collects all product information from the form when the Add button is clicked.
- **createProduct(formData)** for Sends the new product data, including text fields and image file, to the backend system.
- **validateProductForm()** for Checks that required fields such as product name, price, brand, MFG Date, and EXP Date are filled in correctly.
- **cancelAddProduct()** for Sends the admin back to the Product Management Page without adding a new product.
- **initAddProductPage()** for Runs automatically when the page loads, prepares the image preview, validates the form, and handles the submit event.

This script helps administrators add new frozen food products smoothly while previewing the image before saving.

## *3.9 Edit Product Page*

The Edit Product Page (editProduct.html) serves as the dedicated backend interface for administrators to modify existing inventory records within our frozen food web application. It ensures data accuracy and up-to-date catalog information by allowing authorized staff to update prices, shelf-life dates, ingredients, and product imagery.




### *Key Features & UI Components*

**Semantic HTML Structure:** Uses <header>, <nav>, <main>, and <footer> to create a clear, accessible, and well-organized page layout.

**Two-Column Form Layout:** Separates the visual component (image upload and preview) from the textual data entry fields **(.image-col and .form-col).** This optimizes screen space and makes data entry highly intuitive.

**Real-Time Image Preview:** Features a file input coupled with a dynamic preview area **(#previewImage)** powered by a lightweight inline script. It instantly displays the newly selected product photo before the admin clicks update.

**Protected Identifiers:** Includes a specifically locked input field for the Product ID, ensuring the database's primary key cannot be accidentally altered during a routine update.

Footer: Includes contact details, policies, telephone information, and support links.

### *Code Explanation*

**CSS**

CSS is applied through an external stylesheet (/style.css) that defines the website's overall layout and visual structure. For the Edit Product page, it heavily utilizes Flexbox **(display: flex; gap: 40px**;) within the **.add-form-box** to align the image upload column and the data entry column side-by-side. The input fields **(.form-input, .form-textarea)** are styled with custom rounded corners **(border-radius: var(--radius-xxl))** and feature distinct focus states (a combination of border-color changes and box-shadow rings) to clearly indicate which field the administrator is currently typing in.

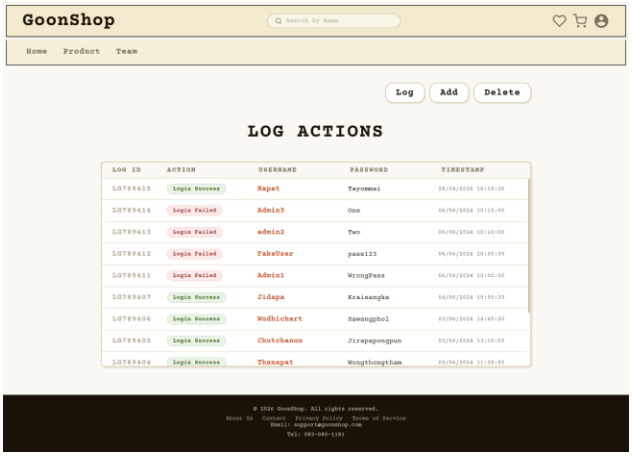
**JS**

- **getProductId()** for Extracts the product ID from the URL so the page knows which product record to edit.
- **loadProductData(id)** for Fetches the existing product information and fills the form fields automatically.
- **previewImage()** for Shows the newly selected product image instantly inside #previewImage before the admin submits the update.
- **handleEditFormSubmit()** for Collects the updated product data from the form and sends it to the backend.
- **updateProduct(id, formData)** for Updates the selected product record, including text fields and image file if changed. cancelEdit() for Returns the admin back to the Product Management Page without saving changes.
- **initEditProductPage()** for Runs automatically when the page loads, gets the product ID, loads the current product data, and prepares the image preview and form submit events.

This script helps administrators edit product information smoothly while preventing the Product ID from being changed accidentally.

## ***3.10 Admin Log Page***

The Admin Log Page (adminLog.html) serves as the system's audit trail and activity monitor for the backend of our frozen food web application. It is designed to provide administrators with transparency regarding system events, tracking which user performed specific actions, the exact time they occurred, and whether those actions were successful or failed.




### ***Key Features & UI Components***

**Semantic HTML Structure:** Uses <header>, <nav>, <main>, and <footer> to create a clear, accessible, and well-organized page layout, consistent with the rest of the admin portal.

**5-Column Audit Grid:** Modifies the standard admin table layout into a specific 5-column grid **(.log-row**) designed to display chronological data efficiently (typically accommodating Log ID, Action, Username, Details, and Timestamp).

**Status Badges:** Utilizes visually distinct, color-coded badges (**.log-success and .log-failed)** to immediately communicate the outcome of system events (e.g., green for successful database writes, red for errors or unauthorized attempts).

**Footer:** Includes contact details, policies, telephone information, and support links.

***Code Explanation***

**CSS**

CSS is applied through an external stylesheet (/style.css) that defines the website's overall layout and visual structure. For the Log Admin page, it relies heavily on CSS Grid overrides. The stylesheet specifically detects the presence of **#logTable** and overrides the standard 4-column product table with a dedicated 5-column layout **(grid-template-columns: 140px 1fr 1fr 1fr 200px)**. It also customizes the typography for technical data, using a fixed-width monospace font **(font-family: var(--font-mono, 'Courier Prime', monospace))** for timestamps to ensure numbers align perfectly down the column. To differentiate logs from clickable products, it removes the hover arrow **(.log-row::after { display: none !important; })** and sets the cursor to default **(cursor: default !important;).**

**JS**

- **loadLogs()** for Loads all admin log records when the Admin Log Page is opened.
- **fetchLogs()** for Requests log data from the backend or system source and returns it as JSON.
- **renderLogs(logs)** for Dynamically displays each log record inside #logTable using .log-row.
- **createLogRow(log)** for Builds one log row including Log ID, Action, Username, Password/Details, and Timestamp.
- **statusBadge(logAction)** for Applies visual status styling such as .log-success or .log-failed depending on whether the action was successful or failed.
- **formatTimestamp(time)** for Converts the timestamp into a readable date and time format for administrators.
- **showEmptyState()** for Displays a message when no log records are available.
- **initAdminLogPage()** for Runs automatically when the page loads, calls loadLogs(), and prepares the log table display.

This script allows administrators to review system activity clearly and monitor successful or failed actions in the backend.

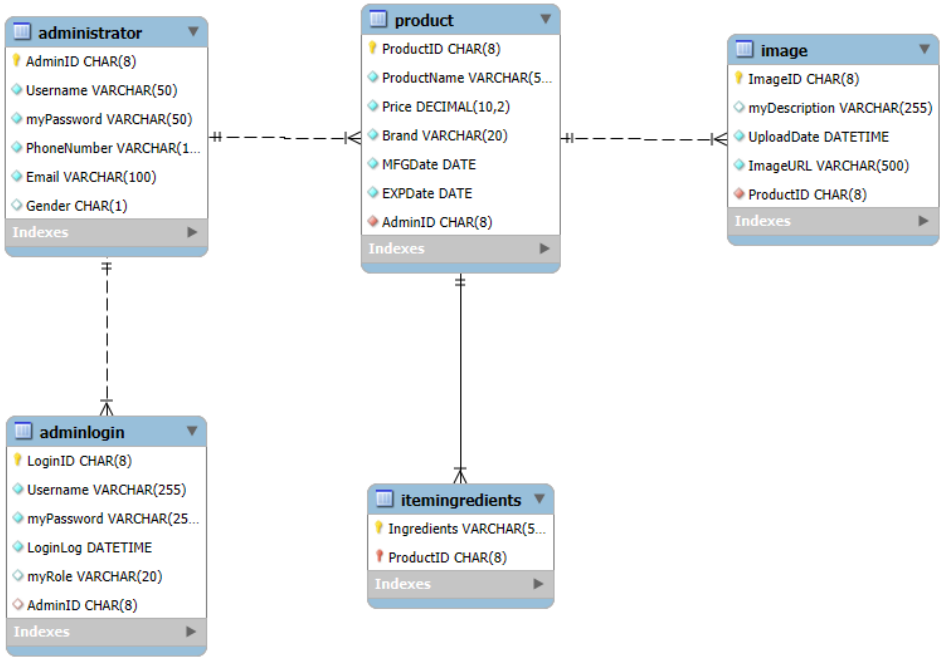
# 4. Details of web services and code

## *4.1 Back-end Overview*

The backend of the GoonShop web application serves as the central processing unit of the system, responsible for handling business logic, managing data, and facilitating communication between the frontend interface and the database. It ensures that all user interactions such as browsing products, searching, and administrative actions are processed efficiently and securely.

## *4.2 Database Overview*

The sec1_gr02_database is designed to support a product management and authentication system. It contains five main tables: Administrator (stores admin details), Product (stores product information), AdminLogin (records login history for auditing), ItemIngredients (stores ingredients linked to each product), and Image (stores product images with URLs). The tables are connected using primary and foreign keys to maintain relationships and ensure data consistency across products, ingredients, images, and admin activities.



## *4.3 Authentication Service*

### 4.3.1 Admin Login

The Authentication Service verifies administrator login credentials to control access to the system. It also records all login attempts in the database for auditing and security tracking purposes.

**Endpoint:** POST /admin/login

**Code and Logical Flow Explanation**

- **Input validation:** Checks whether both **Username** and **myPassword** are provided in the request body. If either is missing, the system returns a 400 Bad Request error.
- **Database query execution:** Executes a parameterized query
  **SELECT * FROM Administrator WHERE BINARY Username = ?**
- **Generate login record ID:** Calls **generateNextId()** to create a unique **LoginID** for recording the login attempt.
- **Credential verification:** Retrieves the admin record and compares the provided password with the stored password. If no admin is found or the password does not match then authentication fails.
- **Logging login attempt:** Regardless of success or failure, the system inserts a record into the **AdminLogin** table including **LoginID, Username, Password, Current timestamp, Role, AdminID** (if successful)
- **Failure response:** If authentication fails, the system returns a 401 Unauthorized response with an **"Invalid credentials"** message.
- **Success response:** If authentication is successful, the system returns a success message along with the corresponding **AdminID**.

**Code Snippet**

```
// POST /admin/login — authenticate an admin and record the attempt
for auditing.
router.post('/admin/login', async (req, res) => {
    const { Username, myPassword } = req.body;

    if (!Username || !myPassword) {
        return res.status(400).send({ error: true, message: 'Missing
username or password' });
    }

    try {
        const [rows]    = await db.promise().query(
            'SELECT * FROM Administrator WHERE BINARY Username = ?',
[Username]
        );
```

```
        const loginID   = await generateNextId('AdminLogin',
'LoginID', 'LG');
        const admin     = rows[0];
        const success   = admin && admin.myPassword === myPassword;

        await db.promise().query(
            `INSERT INTO AdminLogin (LoginID, Username, myPassword,
LoginLog, myRole, AdminID)
             VALUES (?, ?, ?, NOW(), ?, ?)`,
            [loginID, Username, myPassword, success ? 'Admin' : null,
success ? admin.AdminID : null]
        );

        if (!success) return res.status(401).send({ error: true,
message: 'Invalid credentials' });

        res.send({ error: false, message: 'Login success', AdminID:
admin.AdminID });
    } catch (err) {
        console.error(err);
        res.status(500).send({ error: true, message: err.message });
    }
});
```

### 4.3.2 Get login log

For retrieving all login attempt records for monitoring and auditing purposes, allowing administrators to track user access, review login activity, and enhance system security.

**Endpoint:** GET /admin/log

**Code and Logical Flow Explanation**

- **Request received:** The client sends a request to **/admin/log.**
- **Database query execution**: The system executes a query to retrieve all login records from the AdminLogin table ordered by the latest login time (descending) using
- **SELECT LoginID, Username, myPassword, LoginLog, myRole, AdminID**
- **FROM AdminLogin**
- **ORDER BY LoginLog DESC;**
- **Response returned:** The system sends back all login log records as a JSON response with error: false.
- **Error handling**: If a database error occurs, the system returns a **500 Internal Server Error** with the error message.

**Code Snippet**

```
// GET /admin/log — all login attempts, newest first.
router.get('/admin/log', (req, res) => {
    queryOrError(res,
        'SELECT LoginID, Username, myPassword, LoginLog, myRole,
AdminID FROM AdminLogin ORDER BY LoginLog DESC',
        [],
        (rows) => res.send({ error: false, data: rows })
    );
});
```

## *4.4 Product Search Service*

### 4.4.1 No criteria search

For allowing the system to return all available records when no search conditions or filters are specified. This ensures that users can access and view the complete dataset by default, making it easier to browse all information without needing to provide any input criteria. It also supports basic functionality where a full list of data is required, such as displaying all products in the system.

**Endpoint:** GET /products

**Code and Logical Flow Explanation**

- **Request received:** The client sends a request to **/products** without any query parameters.
- **Database query execution:** runs a query to retrieve all records from the Product table using
  **SELECT p.*, i.ImageID, i.ImageURL FROM Product p**
  **LEFT JOIN Image i ON p.ProductID = i.ProductID;**
- **Data processing:** The result is processed using **groupProductImages()** to combine multiple rows into structured product objects with nested image arrays.
- **Response returned:** The system sends back all products in the database as a JSON response with error: false.
- **Error handling:** If any database error occurs, the system returns a 500 Internal Server Error.

**Code Snippet**

```
// GET /products — all products with images.
router.get('/products', (req, res) => {
    const sql = `
        SELECT p.*, i.ImageID, i.ImageURL
        FROM Product p
        LEFT JOIN Image i ON p.ProductID = i.ProductID
    `;
    queryOrError(res, sql, [], (rows) => {
        res.send({ error: false, data: groupProductImages(rows) });
    });
});
```

### 4.4.2 Criteria search

For allowing users to filter and retrieve data based on multiple conditions, improving search flexibility and accuracy. Users can specify all or only some of the available criteria, enabling the system to return more relevant and targeted results instead of the full dataset.

**Endpoint:** GET /products/search

**Code and Logical Flow Explanation**

- **Request received:** The client sends a request to **/products/search** with optional query parameters for minPrice, maxPrice, brand, and ingredient.
- **Database query execution:** The system starts with a base SQL query that selects product data and joins related images and ingredients using
  **SELECT p.*, i.ImageID, i.ImageURL**
  **FROM Product p**
  **LEFT JOIN Image i ON p.ProductID = i.ProductID**
  **LEFT JOIN ItemIngredients ing ON p.ProductID = ing.ProductID**
  **WHERE 1=1**
  **AND p.ProductName LIKE ?**
  **AND p.Price BETWEEN ? AND ?**
  **AND LOWER(p.Brand) = LOWER(?)**
  **AND ing.Ingredients LIKE ?;**
- **Dynamic condition building:** The system checks each provided parameter and dynamically adds conditions to the SQL query using
- minPrice and maxPrice for search within a price range
- brand for search by exact brand (case-insensitive)
- ingredient for search by matching ingredient
- **Data processing:** The result is processed using groupProductImages() to combine rows into structured product objects with nested image arrays.
- **Response returned:** The system returns filtered products as a JSON response with error: false.
- **Error handling**: If any database error occurs, the system returns a 500 Internal Server Error.

**Code Snippet**

```
// GET /products/search — filter by name, price range, brand,
ingredient.
router.get('/products/search', (req, res) => {
    const { name, minPrice, maxPrice, brand, ingredient } =
req.query;

    let sql = `
        SELECT p.*, i.ImageID, i.ImageURL
        FROM Product p
        LEFT JOIN Image i            ON p.ProductID = i.ProductID
```

```
        LEFT JOIN ItemIngredients ing ON p.ProductID = ing.ProductID
        WHERE 1=1
    `;
    const params = [];

    if (name)                 { sql += ' AND p.ProductName LIKE
?';    params.push(`%${name}%`); }
    if (minPrice && maxPrice) { sql += ' AND p.Price BETWEEN ? AND
?'; params.push(minPrice, maxPrice); }
    if (brand)                { sql += ' AND LOWER(p.Brand) =
LOWER(?)'; params.push(brand); }
    if (ingredient)           { sql += ' AND ing.Ingredients LIKE
?';  params.push(`%${ingredient}%`); }

    queryOrError(res, sql, params, (rows) => {
        res.send({ error: false, data: groupProductImages(rows) });
    });
});
```

## *4.5 Product Management Service*

### 4.5.1 Retrieve product all

To provide access to all product records in the system, allowing users or administrators to view the complete list of products along with their details for browsing, management, and decision-making.

**Endpoint:** GET /products

**Code and Logical Flow Explanation**

- **Request received:** The client sends a request to **/products** without any query parameters.
- **Database query execution:** runs a query to retrieve all records from the Product table using
  **SELECT p.*, i.ImageID, i.ImageURL FROM Product p**
  **LEFT JOIN Image i ON p.ProductID = i.ProductID;**
- **Data processing:** The result is processed using **groupProductImages()** to combine multiple rows into structured product objects with nested image arrays.
- **Response returned:**
- The system sends back all products in the database as a JSON response with error: false.
- **Error handling:**
- If any database error occurs, the system returns a 500 Internal Server Error.

**Code Snippet**

```
// GET /products — all products with images.
router.get('/products', (req, res) => {
    const sql = `
        SELECT p.*, i.ImageID, i.ImageURL
        FROM Product p
        LEFT JOIN Image i ON p.ProductID = i.ProductID
    `;
    queryOrError(res, sql, [], (rows) => {
        res.send({ error: false, data: groupProductImages(rows) });
    });
});
```

### 4.5.2 Retrieve product by ID

To retrieve detailed information of a specific product by its unique ID, including its associated images and ingredients, allowing users or administrators to view complete and precise product data.

**Endpoint:** GET /products/:id

**Code and Logical Flow Explanation**

- **Request received:** The client sends a request to /products/:id with a specific ProductID.
- **Database query execution**: The system executes a SQL query to retrieve the product from the Product table, joining with Image and ItemIngredients tables using
  **SELECT p.*, i.ImageID, i.ImageURL, ing.Ingredients**
  **FROM Product p**
  **LEFT JOIN Image i ON p.ProductID = i.ProductID**
  **LEFT JOIN ItemIngredients ing ON p.ProductID = ing.ProductID**
  **WHERE p.ProductID = ?;**
- **Check product existence**: If no records are found, the system returns a **404 Not Found** response.
- **Data processing:** The system creates a product object and Collects unique images into an Images array and Collects unique ingredients into an Ingredients array.
- **Response returned:** The system sends back all products in the database as a JSON response with error: false.
- **Error handling:** If any database error occurs, the system returns a 500 Internal Server Error.

**Code Snippet**

```
// GET /products/:id — single product with images and ingredients.
router.get('/products/:id', (req, res) => {
    const sql = `
        SELECT p.*, i.ImageID, i.ImageURL, ing.Ingredients
        FROM Product p
        LEFT JOIN Image i            ON p.ProductID = i.ProductID
        LEFT JOIN ItemIngredients ing ON p.ProductID = ing.ProductID
        WHERE p.ProductID = ?
    `;
    queryOrError(res, sql, [req.params.id], (rows) => {
        if (rows.length === 0) return res.status(404).send({ error:
true, message: 'Not found' });

        const { ImageID, ImageURL, ...fields } = rows[0];
        const product = { ...fields, Images: [], Ingredients: [] };

        for (const row of rows) {
            if (row.ImageID && !product.Images.find((img) =>
img.ImageID === row.ImageID)) {
                product.Images.push({ ImageID: row.ImageID, ImageURL:
row.ImageURL });
            }
            if (row.Ingredients &&
!product.Ingredients.includes(row.Ingredients)) {
                product.Ingredients.push(row.Ingredients);
            }
        }

        res.send({ error: false, data: product });
    });
});
```

### 4.5.3 Insert Product

To add a new product into the system, including its details, ingredients, and optional image, allowing administrators to expand and manage the product inventory.

**Endpoint:** POST /products

**Code and Logical Flow Explanation**

- **Request received:** The client sends a POST /products request with product details in the request body and optionally an image file.
- **Generate Product ID:** The system generates a new unique ProductID using **generateNextId()**.
- **Database query execution (Insert Product)**: The system executes the following SQL query to store product information using
  **INSERT INTO Product VALUES (?, ?, ?, ?, ?, ?, ?);**
- **Insert ingredients (if provided)**: If ingredients are included, the system executes using
  **INSERT INTO ItemIngredients (Ingredients, ProductID) VALUES ?;**
- **Response returned:** The system returns a success message along with the newly created ProductID.
- **Error handling:** If any step fails, the system returns a 500 Internal Server Error with the error message.

**Code Snippet**

```
// POST /products — create a product with optional image upload.
router.post('/products', upload.single('image'), async (req, res) =>
{
    const { ProductName, Price, Brand, MFGDate, EXPDate, AdminID,
Ingredients } = req.body;

    try {
        const newProductID = await generateNextId('Product',
'ProductID', 'PD');

        await db.promise().query(
            'INSERT INTO Product VALUES (?, ?, ?, ?, ?, ?, ?)',
            [newProductID, ProductName, Price, Brand, MFGDate,
EXPDate, AdminID]
        );

        if (Ingredients) await replaceIngredients(Ingredients,
newProductID);

        if (req.file) {
            const { secure_url } = await
uploadToCloudinary(req.file.buffer);
            await saveImage(secure_url, newProductID);
```

```
        }

        res.send({ error: false, message: 'Product created',
ProductID: newProductID });
    } catch (err) {
        console.error(err);
        res.status(500).send({ error: true, message: err.message });
    }
});
```

### 4.5.4 Update Product

To modify and update existing product information in the system, including product details, ingredients, and image, allowing administrators to keep product data accurate and up to date.

**Endpoint:** PUT /products/:id

**Code and Logical Flow Explanation**

- **Request received:** The client sends a PUT /products/:id request with the ProductID in the URL and updated product data (and optionally an image file) in the request body.
- **Database query execution (Update Product):** The system updates the product information in the Product table using
  **UPDATE Product**
  **SET ProductName=?, Price=?, Brand=?, MFGDate=?, EXPDate=?, AdminID=?**
  **WHERE ProductID=?;**
- **Response returned:** The system returns a success message indicating the product has been updated.
- **Error handling:** If any database or upload error occurs, the system returns a 500 Internal Server Error with the error message.

**Code Snippet**

```
// PUT /products/:id — update product fields, ingredients, and
optionally the image.
router.put('/products/:id', upload.single('image'), async (req, res)
=> {
    const { id } = req.params;
    const { ProductName, Price, Brand, MFGDate, EXPDate, AdminID,
Ingredients } = req.body;

    try {
        await db.promise().query(
            'UPDATE Product SET ProductName=?, Price=?, Brand=?,
MFGDate=?, EXPDate=?, AdminID=? WHERE ProductID=?',
```

```
            [ProductName, Price, Brand, MFGDate, EXPDate, AdminID,
id]
        );

        if (Ingredients) await replaceIngredients(Ingredients, id);
        if (req.file)    await replaceImage(req.file.buffer, id);

        res.send({ error: false, message: 'Updated' });
    } catch (err) {
        console.error(err);
        res.status(500).send({ error: true, message: err.message });
    }
});
```

### 4.5.5 Delete Product

To remove a product and all its related data (such as images and ingredients) from the system, ensuring that the product is completely deleted from the database and no longer accessible or displayed.

**Endpoint:** DELETE /products/:id

**Code and Logical Flow Explanation**

- **Request received:** The client sends a DELETE /products/:id request with a specific ProductID.
- **Database query execution (delete related data first):** The system first removes all related records to maintain data integrity using
  **DELETE FROM Image WHERE ProductID = ?;**
  **DELETE FROM ItemIngredients WHERE ProductID = ?;**
- **Database query execution (delete product):** After related data is removed, the product itself is deleted using
  **DELETE FROM Product WHERE ProductID = ?;**
- **Response returned:** The system returns a success message confirming the product has been deleted.
- **Error handling:** If any database operation fails, the system returns a 500 Internal Server Error with the error message.

**Code Snippet**

```
// DELETE /products/:id — remove product and all related records.
router.delete('/products/:id', async (req, res) => {
    const { id } = req.params;

    try {
        await db.promise().query('DELETE FROM Image           WHERE
ProductID = ?', [id]);
        await db.promise().query('DELETE FROM ItemIngredients  WHERE
ProductID = ?', [id]);
        await db.promise().query('DELETE FROM Product          WHERE
ProductID = ?', [id]);

        res.send({ error: false, message: 'Deleted' });
    } catch (err) {
        console.error(err);
        res.status(500).send({ error: true, message: err.message });
    }
});
```

# 5. Web Service Testing Results

## ***5.1 Testing Tool***

Postman is used as a testing tool to verify and validate the functionality of RESTful API endpoints in the system. It allows developers to send HTTP requests, simulate client behavior, and inspect server responses without needing a frontend interface.

## ***5.2 Authentication Service Testing***

***Test Case 1:*** *Insert all login log (Valid login)*

status: 200 OK
method: post
URL: http://localhost:3030/admin/login
body: raw JSON


```
{
   "Username": "Wudhichart",
   "myPassword": "Sawangphol"
}
Result: {
   "error": false,
   "message": "Login success",
   "AdminID": "AD789406"
}
```


***Test Case 2:*** *Insert all login log (Wrong Username / Password)*

status: 401 Unauthorized
method: post
URL: http://localhost:3030/admin/login
body: raw JSON


```
{
   "Username": "NonAdmin",
   "myPassword": "One"
}
Result: {
   "error": true,
   "message": "Invalid credentials"
}
```

***Test Case 3:*** *Retrieve all admin login information (Normal case)*

status: 200 OK
method: get
URL: http://localhost:3030/admin/log
body: none
Result: {
"error": false,
"data": [
{
"LoginID": "LG789416",
"Username": "NonAdmin",
"myPassword": "One",
"LoginLog": "2026-04-28 19:19:19",
"myRole": null,
"AdminID": null
},
{
"LoginID": "LG789415",
"Username": "Wudhichart",
"myPassword": "Sawangphol",
"LoginLog": "2026-04-28 19:14:43",
"myRole": "Admin",
"AdminID": "AD789406"
},
…
]
}

***Test Case 4:*** *Retrieve all admin login information (Database table missing)*
status: 500 Internal Server Error
method: get
URL: http://localhost:3030/admin/log
body: none
Result: {
"error": true,
"message": "Table 'sec1_gr02_database.adminlogin' doesn't exist"
}

## ***5.3 Product Search Service Testing***

***Test Case 5:*** *Retrieve a product (Search by name)*
**status:** 200 OK
**method:** get
**URL:** http://localhost:3030/products/search?name=chicken wing
**body:** none
**Result:** {
"error": false,
"data": [
{
"ProductID": "PD789407",
"ProductName": "Chicken Wing",
"Price": "375.00",
"Brand": "Aro",
"MFGDate": "2024-01-01",
"EXPDate": "2025-01-01",
"AdminID": "AD789407",
"Images": [
{
"ImageID": "IM789407",
"ImageURL": "https://cdn-app.cp-cmpd.com/images/cpknow/f66da48c437619ac84b5be909096ef46.png"
}
]
}
]
}

***Test Case 6:*** *Retrieve a product (Criteria search)*

status: 200 OK
method: get
URL:
http://localhost:3030/products/search?minPrice=100&maxPrice=500&brand=CP&ingredient=Pork
body: none
Result: {


```
   "error": false,
   "data": [
      {
         "ProductID": "PD789402",
         "ProductName": "Frozen Pork",
         "Price": "150.00",
         "Brand": "CP",
         "MFGDate": "2024-01-01",
         "EXPDate": "2025-01-01",
         "AdminID": "AD789402",
         "Images": [
            {
               "ImageID": "IM789402",
               "ImageURL": "https://images.mango-prod.siammakro.cloud/SOURCE/7683200ac13945eeb50f6ed1e0ce6307?eo-img.resize=w%2F1080&eo-img.format=webp"
            }
         ]
      },
      …
   ]
}
```

## ***5.4 Product Management Service***

***Test Case 7:*** *Retrieve all products (Normal case)*

status: 200 OK
method: get
URL: http://localhost:3030/products
body: none
Result: {


```
   "error": false,
   "data": [
      {
         "ProductID": "PD789401",
         "ProductName": "Crispy Chicken",
         "Price": "120.00",
         "Brand": "CP",
         "MFGDate": "2024-01-01",
         "EXPDate": "2025-01-01",
         "AdminID": "AD789401",
         "Images": [
            {
               "ImageID": "IM789401",
               "ImageURL": "https://images.mango-prod.siammakro.cloud/product-
images/6974745477315-983ca1d7-79e7-4038-b53b-65af394f3f3c.jpeg?eo-
img.resize=w%2F1080&eo-img.format=webp"
            }
         ]
      },
      …
   ]
}
```


***Test Case 8:*** *Retrieve all products (Database table missing)*

status: 500 Internal Server Error
method: get
URL: http://localhost:3030/products
body: none
Result: {


```
   "error": true,
   "message": "Table 'sec1_gr02_database.product' doesn't exist"
}
```

***Test Case 9:*** *Retrieve a product by ID (Normal case)*

status: 200 OK
method: get
URL: http://localhost:3030/products/PD789410
body: none
Result: {


```
   "error": false,
   "data": {
      "ProductID": "PD789410",
      "ProductName": "Frozen Donut shrimp",
      "Price": "160.00",
      "Brand": "CP",
      "MFGDate": "2024-01-01",
      "EXPDate": "2025-01-01",
      "AdminID": "AD789403",
      "Ingredients": [
         "Egg",
         "Flour",
         "Shrimp"
      ],
      "Images": [
         {
            "ImageID": "IM789410",
            "ImageURL": "https://down-th.img.susercontent.com/file/th-11134207-81ztq-
mm2ajjojyqz356.webp"
         }
      ]
   }
}
```


***Test Case 10:*** *Retrieve a product by ID (Invalid ID)*

status: 404 Not Found
method: get
URL: http://localhost:3030/products/XX000000
body: none
Result: {


```
   "error": true,
   "message": "Not found"
}
```

***Test Case 11:*** *Insert a product (Normal case)*

status: 200 OK
method: post
URL: http://localhost:3030/products
body: form-data

(Key) : (Value)
ProductName : Frozen Burger
Price : 350
Brand : Burger Queen
MFGDate : 2026-01-01
EXPDate : 2027-01-01
AdminID : AD789406
Ingredients : Chicken, Bun, Sauce
Image (File) : (upload image file)

Result: {
"error": false,
"message": "Product created",
"ProductID": "PD789411"
}

***Test Case 12:*** *Insert a product (Missing field)*

status: 500 Internal Server Error
method: post
URL: http://localhost:3030/products
body: form-data

(Key) : (Value)
ProductName : Frozen Lamb
Price : 100
Brand : Lamb Queen
AdminID : AD789407
Image (File) : (upload image file)

Result: {
"error": true,
"message": "Column 'Brand' cannot be null"
}

***Test Case 13:*** *Update a product (Normal case)*

status: 200 OK
method: put
URL: http://localhost:3030/products/PD789401
body: form-data
(Key) : (Value)
ProductName : Chicken Bites
Price : 789
Brand : Aro
MFGDate : 2024-01-01
EXPDate : 2025-01-01
AdminID : AD789403
Ingredients : Chicken, Flour, Oil, Salt
image (File) : (upload image file)
Result: {
"error": false,
"message": "Updated"
}

***Test Case 14:*** *Update a product (Missing field)*

status: 500 Internal Server Error
method: put
URL: http://localhost:3030/products/PD789401
body: form-data
(Key) : (Value)
ProductName : Chicken Bites
Price : 789
Ingredients : Chicken, Flour, Oil, Salt
image (File) : (upload image file)
Result: {
"error": true,
"message": "Column 'Brand' cannot be null"
}

***Test Case 15:*** *Delete a product (Normal case)*

status: 200 OK
method: delete
URL: http://localhost:3030/products/PD789410
body: none
Result: {

```
   "error": false,
   "message": "Deleted"
}
```


***Test Case 16:*** *Delete a products (Database table missing)*

status: 500 Internal Server Error
method: delete
URL: http://localhost:3030/products/PD789410
body: none
Result: {

```
   "error": true,
   "message": "Table 'sec1_gr02_database.image' doesn't exist"
}
```


**Test Case 17:** Retrieve all ingredients (Normal case)

status: 200 OK
method: get
URL: http://localhost:3030/ingredients
body: none
Result: {

```
   "error": false,
   "data": [
      "Beef",
      "Cabbage",
      "Cheese",
         …
   ]
}
```

***Test Case 18:*** *Retrieve all ingredients (Database table missing)*

status: 500 Internal Server Error
method: get
URL: http://localhost:3030/ingredients
body: none
Result: {
"error": true,
"message": "Table 'sec1_gr02_database.itemingredients' doesn't exist"
}